

torch.escape-hatch#

<https://pytorch.org/docs/stable/generated/exportdb/torch.escape-hatch.html>

Description:

Tags: torch.escape-hatch Support Level: SUPPORTED Original source code:

Code Examples

```
# mypy: allow-untyped-defs
import torch
import torch._dynamo as torchdynamo

class AssumeConstantResult(torch.nn.Module):
    """
    Applying `assume_constant_result` decorator to burn make
    non-tracable code as constant.
    """

    @torchdynamo.assume_constant_result
    def get_item(self, y):
        return y.int().item()

    def forward(self, x, y):
        return x[: self.get_item(y)]

example_args = (torch.randn(3, 2), torch.tensor(4))
tags = {"torch.escape-hatch"}
model = AssumeConstantResult()

torch.export.export(model, example_args)
```

ExportedProgram:

```
class GraphModule(torch.nn.Module):  
    def forward(self, x: "f32[3, 2]", y: "i64[]"):  
        slice_1: "f32[3, 2]" =  
torch.ops.aten.slice.Tensor(x, 0, 0, 4); x = None  
        return (slice_1,)
```

Graph signature:

inputs

x: USER_INPUT

y: USER_INPUT

outputs

slice_1: USER_OUTPUT

Range constraints: {}

```

# mypy: allow-untyped-defs
import torch

class ConstrainAsSizeExample(torch.nn.Module):
    """
    If the value is not known at tracing time, you can provide
    hint so that we
    can trace further. Please look at torch._check and
    torch._check_is_size APIs.
    torch._check_is_size is used for values that NEED to be used
    for constructing
    tensor.
    """

    def forward(self, x):
        a = x.item()
        torch._check_is_size(a)
        torch._check(a <= 5)
        return torch.zeros((a, 5))

example_args = (torch.tensor(4),)
tags = {
    "torch.dynamic-value",
    "torch.escape-hatch",
}
model = ConstrainAsSizeExample()

torch.export.export(model, example_args)

```

ExportedProgram:

```
class GraphModule(torch.nn.Module):
    def forward(self, x: "i64[]"):
        item: "Sym(u0)" =
torch.ops.aten.item.default(x);  x = None

        #
        sym_constrain_range_for_size_default =
torch.ops.aten.sym_constrain_range_for_size.default(item);
sym_constrain_range_for_size_default = None

        ge_1: "Sym(u0 >= 0)" = item >= 0
        _assert_scalar_default =
torch.ops.aten._assert_scalar.default(ge_1, "Runtime assertion
failed for expression u0 >= 0 on node 'ge_1'");  ge_1 =
_assert_scalar_default = None
        le_1: "Sym(u0 <= 5)" = item <= 5
        _assert_scalar_default_1 =
torch.ops.aten._assert_scalar.default(le_1, "Runtime assertion
failed for expression u0 <= 5 on node 'le_1'");  le_1 =
_assert_scalar_default_1 = None

        zeros: "f32[u0, 5]" =
torch.ops.aten.zeros.default([item, 5], device =
device(type='cpu'), pin_memory = False);  item = None
        return (zeros,)
```

Graph signature:

```
# inputs
x: USER_INPUT

# outputs
zeros: USER_OUTPUT
```

Range constraints: {u0: VR[0, 5], u1: VR[0, 5]}

```

# mypy: allow-untyped-defs
import torch

class ConstrainAsValueExample(torch.nn.Module):
    """
    If the value is not known at tracing time, you can provide
    hint so that we
    can trace further. Please look at torch._check and
    torch._check_is_size APIs.
    torch._check is used for values that don't need to be used
    for constructing
    tensor.
    """

    def forward(self, x, y):
        a = x.item()
        torch._check(a >= 0)
        torch._check(a <= 5)

        if a < 6:
            return y.sin()
        return y.cos()

example_args = (torch.tensor(4), torch.randn(5, 5))
tags = {
    "torch.dynamic-value",
    "torch.escape-hatch",
}
model = ConstrainAsValueExample()

torch.export.export(model, example_args)

```

ExportedProgram:

```
class GraphModule(torch.nn.Module):
    def forward(self, x: "i64[]", y: "f32[5, 5]"):
        item: "Sym(u0)" =
torch.ops.aten.item.default(x); x = None
        ge_1: "Sym(u0 >= 0)" = item >= 0
        _assert_scalar_default =
torch.ops.aten._assert_scalar.default(ge_1, "Runtime assertion
failed for expression u0 >= 0 on node 'ge_1'"); ge_1 =
_assert_scalar_default = None
        le_1: "Sym(u0 <= 5)" = item <= 5; item = None
        _assert_scalar_default_1 =
torch.ops.aten._assert_scalar.default(le_1, "Runtime assertion
failed for expression u0 <= 5 on node 'le_1'"); le_1 =
_assert_scalar_default_1 = None

        sin: "f32[5, 5]" =
torch.ops.aten.sin.default(y); y = None
        return (sin,)
```

Graph signature:

```
# inputs
x: USER_INPUT
y: USER_INPUT

# outputs
sin: USER_OUTPUT
```

Range constraints: {u0: VR[0, 5], u1: VR[0, 5]}

Notes & Warnings

NOTE

Note Tags: torch.escape-hatch Support Level: SUPPORTED

NOTE

Note Tags: torch.dynamic-value, torch.escape-hatch Support Level: SUPPORTED

NOTE

Note Tags: torch.dynamic-value, torch.escape-hatch Support Level: SUPPORTED

Detailed Documentation

assume_constant_result#

Original source code:

constrain_as_size_example#

Original source code:

constrain_as_value_example#

Original source code:

Tags: torch.escape-hatch

Support Level: SUPPORTED

Original source code:

Tags: torch.dynamic-value, torch.escape-hatch

Support Level: SUPPORTED

Original source code:

Tags: torch.dynamic-value, torch.escape-hatch

Support Level: SUPPORTED

Original source code: